Phase -1.7 — Architecture Decision Records (ADR)

Objective

Every production system is the result of hundreds of engineering decisions.

One team may choose PostgreSQL.

Another chooses MongoDB.

One team builds a monolith.

Another chooses microservices.

Neither choice is inherently correct.

Each decision represents a trade-off.

Architecture Decision Records (ADRs) document:

- the problem,
- available alternatives,
- evaluation criteria,
- chosen solution,
- consequences of that decision.

ADRs are maintained because future developers should understand **why** a decision was made rather than guessing from the implementation.

For this project, every major architectural choice will be recorded before implementation begins.

What is an ADR?

An Architecture Decision Record is a lightweight document describing an important technical decision.

Every ADR follows the same template.

Status

Accepted

Context

What problem are we solving?

Options Considered

Every reasonable alternative.

Decision

Which option was selected?

Consequences

Advantages

Disadvantages

Future considerations

ADR-001 — Modular Monolith vs Microservices

Status

Accepted

Context

The application could be designed either as a modular monolith or as multiple independent services.

Options Considered

Option 1

Modular Monolith

Option 2

Microservices

Decision

Use a **Modular Monolith**.

Rationale

Advantages

- Easier to develop.
- Easier debugging.
- Single deployment.
- Single repository.
- Lower operational complexity.
- Ideal for learning backend engineering.

Disadvantages

- Larger codebase over time.
- Independent scaling is not possible.

Future Evolution

The internal module boundaries will make it possible to extract services later if required.

ADR-002 — Node.js + TypeScript

Status

Accepted

Context

Several backend platforms were considered.

Options Considered

- Java Spring Boot
- Go
- .NET
- Python

Decision

Node.js with TypeScript.

Rationale

Advantages

- Excellent ecosystem.
- Fast development.
- Strong TypeScript support.
- Large industry adoption.
- Familiar technology stack.

Disadvantages

- Single-threaded execution.
- CPU-intensive workloads require additional strategies.

ADR-003 — Express vs Other Frameworks

Options Considered

- Express
- Fastify
- NestJS
- Hono

Decision

Express.

Why

Express exposes HTTP concepts directly.

Learning middleware, routing, request lifecycles and error handling is easier because less framework abstraction exists.

Once these concepts are understood, transitioning to frameworks such as NestJS becomes significantly easier.

ADR-004 — PostgreSQL vs NoSQL

Options Considered

- PostgreSQL
- MySQL
- MongoDB
- Cassandra

Decision

PostgreSQL.

Why

The project contains:

- transactions,
- relationships,
- constraints,
- concurrency control,
- ACID guarantees.

These are strengths of relational databases.

Trade-offs

MongoDB offers schema flexibility.

However, maintaining transactional consistency for bookings is simpler with PostgreSQL.

ADR-005 — Prisma vs Raw SQL

Options Considered

- Prisma
- TypeORM
- Drizzle
- Raw SQL

Decision

Prisma.

Why

Prisma provides:

- Strong type safety.
- Migration management.
- Excellent developer experience.
- Readable schema definitions.
- Simplified onboarding.

Raw SQL will still be used when it provides better control or performance.

The project intentionally combines both approaches.

ADR-006 — JWT Authentication

Options Considered

- Server Sessions
- JWT
- OAuth
- API Keys

Decision

JWT with rotating Refresh Tokens.

Why

Advantages

- Stateless servers.
- Horizontal scalability.
- Widely used in REST APIs.
- Suitable for distributed systems.

Refresh token rotation improves security while preserving user experience.

ADR-007 — REST vs GraphQL

Options Considered

- REST
- GraphQL
- gRPC

Decision

REST.

Why

REST remains the dominant API style for backend services.

It is simpler to understand, easier to document, and aligns well with the educational goals of this project.

GraphQL introduces additional complexity that is unnecessary for this domain.

ADR-008 — Redis

Decision

Use Redis as an in-memory infrastructure component.

Responsibilities

- Cache
- BullMQ backend
- Temporary state
- Future distributed locking

Redis is intentionally **not** treated as the primary database.

ADR-009 — Docker

Decision

Containerize every infrastructure dependency.

Eventually containerize the application itself.

Why

Advantages

- Identical environments.
- Simplified onboarding.
- Repeatable builds.
- Production parity.
- Easy local setup.

This minimizes the classic "works on my machine" problem.

ADR-010 — Global Error Handling

Decision

Use centralized exception handling.

Why

Controllers should only coordinate requests.

Formatting error responses, logging exceptions, and mapping domain errors to HTTP responses should occur in one location.

This ensures:

- Consistent API responses.
- Better logging.
- Reduced duplication.
- Easier maintenance.

ADR-011 — Layered Modular Architecture

Decision

Adopt a layered architecture organized around business modules.

Layers

- Router
- Controller
- Service
- Repository
- Database

Why

Each layer has a single responsibility.

Dependencies always flow downward.

Business logic remains isolated from infrastructure concerns.

ADR-012 — Development Environment Strategy

Decision

Use environment-based configuration rather than environment-specific code.

Development

- Local Docker
- Hot reload
- Debug logging
- Swagger enabled

Production

- Compiled JavaScript
- Structured logging
- Secret manager
- Monitoring enabled
- Debug endpoints disabled

Why

The application should behave differently through configuration rather than conditional code.

This keeps deployments predictable and reduces configuration drift.

Architecture Principles

Every future architectural decision should satisfy these principles.

1. Prefer simplicity over cleverness.
2. Favor explicitness over hidden behavior.
3. Keep business logic framework-independent.
4. Design for maintainability before optimization.
5. Make infrastructure replaceable.

6. Prefer composition over inheritance.
7. Fail fast when configuration is invalid.
8. Keep modules loosely coupled.
9. Keep APIs consistent.
10. Optimize only after measurement.

Deliverables

By the completion of this phase, every major architectural decision has been documented before implementation.

Future contributors should be able to answer not only **what** the system does, but also **why** each technology, framework, and architectural pattern was selected.

The next phase, **Phase -1.8 — API Design & REST Contract**, will formalize the public interface of the application using the decisions recorded in these ADRs.